

1.3 Sección 3 – Módulos y Paquetes

Deslizar para comenzar ↓

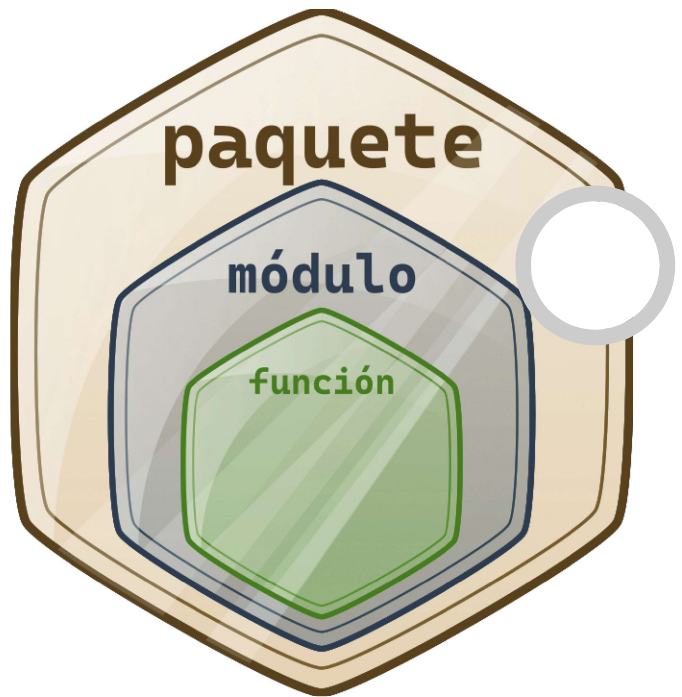
1.3.1 ¿Qué es un paquete?

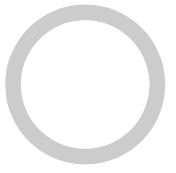
Escribir tus propios módulos no difiere mucho de escribir scripts comunes.

Existen algunos aspectos específicos que se deben tomar en cuenta, pero definitivamente no es algo complicado. Lo verás pronto.

Resumamos algunos aspectos importantes:

- Un **módulo es un contenedor lleno de funciones** - puedes empaquetar tantas funciones como desees en un módulo y distribuirlo por todo el mundo.
- Por supuesto, no es una buena idea mezclar funciones con diferentes áreas de aplicación dentro de un módulo (al igual que en una biblioteca: nadie espera que los trabajos científicos se incluyan entre los cómics), así que se deben agrupar las funciones cuidadosamente y asignar un nombre claro e intuitivo al módulo que las contiene (por ejemplo, no le des el nombre `videojuegos` a un módulo que contiene funciones destinadas a particionar y formatear discos duros).
- Crear muchos módulos puede causar desorden: tarde que temprano querrás **agrupar tus módulos** de la misma manera que previamente has agrupado funciones: ¿Existe un contenedor más general que un módulo?
- Sí lo hay, es un **paquete**: en el mundo de los módulos, un paquete juega un papel similar al de una carpeta o directorio en el mundo de los archivos.





1.3.2 Tu primer módulo

Paso 1

En esta sección, trabajarás localmente en tu máquina. Comencemos desde cero. Crea un archivo vacío, de la siguiente manera:

```
1 |  
2 |
```

module.py

Se necesitan dos archivos para realizar estos experimentos. Uno de ellos será el *módulo* en sí. Está vacío ahora. No te preocupes, lo vas a llenar con código pronto.

El archivo lleva por nombre `module.py`. No muy creativo, pero es simple y claro.

Paso 2

El segundo archivo contiene el código que utiliza el nuevo módulo. Su nombre es `main.py`. Su contenido es muy breve hasta ahora:

```
1 | import module  
2 |
```

main.py

Nota: ambos archivos deben estar ubicados en la misma carpeta. Te recomendamos crear una carpeta nueva y vacía para ambos archivos. Esto hará que las cosas sean más fáciles.

Inicia el IDLE (o cualquier otro que prefieras) y ejecuta el archivo `main.py`. ¿Qué es lo que ves?

No deberías ver nada. Esto significa que Python ha importado con éxito el contenido del archivo `module.py`.

No importa que el módulo esté vacío por ahora. El primer paso ya está hecho, pero antes de dar el siguiente paso, queremos que eches un vistazo a la carpeta en la que se encuentran ambos archivos.

¿Notas algo interesante?

Ha aparecido una nueva subcarpeta, ¿puedes verla? Su nombre es `__pycache__`. Echa un vistazo adentro. ¿Qué es lo que ves?

Hay un archivo llamado (más o menos) `module.cpython-xy.pyc` donde `x` y `y` son dígitos derivados de tu versión de Python (por ejemplo, serán 3 y 8 si utilizas Python 3.8).

El nombre del archivo es el mismo que el de tu módulo. La parte posterior al primer punto dice qué implementación de Python ha creado el archivo (CPython) y su número de versión. La última parte (`.pyc`) viene de las palabras *Python* y *compilado*.

Puedes mirar dentro del archivo: el contenido es completamente ilegible para los humanos. Tiene que ser así, ya que el archivo está destinado solo para uso de Python.

Cuando Python importa un módulo por primera vez, **traduce el contenido a una forma algo compilada**.

El archivo no contiene código en lenguaje máquina: es **código semi-compilado** interno de Python, listo para ser ejecutado por el intérprete de Python. Como tal archivo no requiere tantas comprobaciones como las de un archivo fuente, la ejecución comienza más rápido y también se ejecuta más rápido.

Gracias a eso, cada importación posterior será más rápida que interpretar el código fuente desde cero.

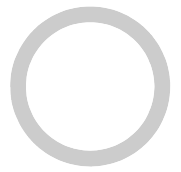
Python puede verificar si el archivo fuente del módulo ha sido modificado (en este caso, el archivo `.pyc` será reconstruido) o no (cuando el archivo `.pyc` pueda ser ejecutado al instante). Este proceso es completamente automático y transparente, no tiene que ser tomando en cuenta.

Paso 3

Ahora hemos puesto algo en el archivo del módulo:

```
1 | print("Me gusta ser un módulo.")
2 |
```

module.py



¿Puedes notar alguna diferencia entre un módulo y un script ordinario? No hay ninguna hasta ahora.

Es posible ejecutar este archivo como cualquier otro script. Pruébalo por ti mismo.

¿Qué es lo que pasa? Deberías de ver la siguiente línea dentro de tu consola:

Output

```
Me gusta ser un módulo.
```

Paso 4

Volvamos al archivo `main.py` :

```
1 | import module
2 |
```

main.py

Ejecuta el archivo. ¿Qué ves? Con suerte, verás algo como esto:

Output

```
Me gusta ser un módulo.
```

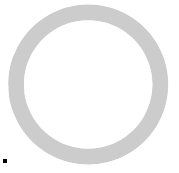
¿Qué significa realmente?

Cuando un módulo es importado, su contenido es **ejecutado implícitamente por Python**. Le da al módulo la oportunidad de inicializar algunos de sus aspectos internos (por ejemplo, puede asignar a algunas variables valores útiles).

Nota: la **inicialización se realiza sólo una vez**, cuando se produce la primera importación, por lo que las asignaciones realizadas por el módulo no se repiten innecesariamente.

Imagina el siguiente contexto:

- Existe un módulo llamado `mod1`.
- Existe un módulo llamado `mod2` el cual contiene la instrucción `import mod1`.
- Hay un archivo principal que contiene las instrucciones `import mod1` y `import mod2`.



A primera vista, se puede pensar que `mod1` será importado dos veces, afortunadamente, **solo se produce la primera importación**. Python recuerda los módulos importados y omite silenciosamente todas las importaciones posteriores.

Paso 5

Python puede hacer mucho más que solo importar el módulo. También crea una variable llamada `__name__`.

Además, cada archivo fuente usa su propia versión separada de la variable, no se comparte entre módulos.

Te mostraremos como usarlo. Modifica el módulo un poco:

```
1 | print("Me gusta ser un módulo.")
2 | print(__name__)
```

`module.py`

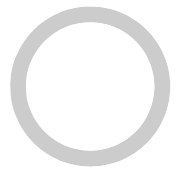
Ahora ejecuta el archivo `module.py`. Deberías ver las siguientes líneas:

Output

```
Me gusta ser un módulo
__main__
```

Ahora ejecuta el archivo `main.py`. ¿Y? ¿Ves lo mismo que nosotros?

```
Me gusta ser un módulo  
module
```



Podemos decir que:

- Cuando se ejecuta un archivo directamente, su variable `__name__` se establece a `__main__`;
- Cuando un archivo se importa como un módulo, su variable `__name__` se establece al nombre del archivo (excluyendo a `.py`)

Paso 6

Así es como puedes hacer uso de la variable `__main__` para detectar el contexto en el cual se activó tu código:

```
1 | if __name__ == "__main__":  
2 |     print("Prefiero ser módulo.")  
3 | else:  
4 |     print("Me gusta ser un módulo.")  
5 |
```

module.py

Sin embargo, existe una forma más inteligente de utilizar la variable. Si escribes un módulo lleno de varias funciones complejas, puedes usarla para colocar una serie de pruebas para verificar si las funciones trabajan correctamente.

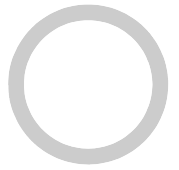
Cada vez que modifiques alguna de estas funciones, simplemente puedes ejecutar el módulo para asegurarte de que sus enmiendas no estropeen el código. Estas pruebas se omitirán cuando el código se importe como un módulo.

Paso 7

Este módulo contendrá dos funciones simples, y si deseas saber cuantas veces se han invocado las funciones, necesitas un contador inicializado en cero cuando se importe el módulo.

Puedes hacerlo de esta manera:

```
1 | counter = 0
2 |
3 | if __name__ == "__main__":
4 |     print("Prefiero ser un módulo.")
5 | else:
6 |     print("Me gusta ser un módulo.")
7 |
```



module.py

Paso 8

El introducir tal variable es absolutamente correcto, pero puede causar importantes **efectos secundarios** que debes tomar en cuenta.

Analiza el archivo modificado `main.py` :

```
1 | import module
2 | print(module.counter)
3 |
```

main.py

Como puedes ver, el archivo principal intenta acceder a la variable de contador del módulo. ¿Es esto legal? Sí lo es. ¿Es utilizable? Claro. ¿Es seguro?

Eso depende: si confías en los usuarios de tu módulo, no hay problema; sin embargo, es posible que no desees que el resto del mundo vea tu **variable personal o privada**.

A diferencia de muchos otros lenguajes de programación, Python no tiene medios para permitirte ocultar tales variables a los ojos de los usuarios del módulo.

Solo puedes informar a tus usuarios que esta es tu variable, que pueden leerla, pero que no deben modificarla bajo ninguna circunstancia.

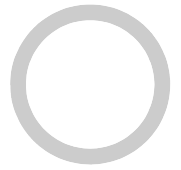
Esto se hace anteponiendo al nombre de la variable `_` (un guión bajo) o `__` (dos guiones bajos), pero recuerda, es solo un **acuerdo**. Los usuarios de tu módulo pueden obedecerlo o no.

Nosotros por supuesto, lo respetaremos. Ahora pongamos dos funciones en el módulo: evaluarán la suma y el producto de los números recopilados en una lista.

Además, agreguemos algunos adornos allí y eliminemos los restos superfluos.

Paso 9

Bueno. Escribamos un código nuevo en nuestro archivo `module.py`. El módulo actualizado está listo aquí:



```
1  #!/usr/bin/env python3
2
3  """ module.py - an example of a Python module """
4
5  __counter = 0
6
7
8  def suml(the_list):
9      global __counter
10     __counter += 1
11     the_sum = 0
12     for element in the_list:
13         the_sum += element
14     return the_sum
15
16
17 def prodl(the_list):
18     global __counter
19     __counter += 1
20     prod = 1
21     for element in the_list:
22         prod *= element
23     return prod
24
25
26 if __name__ == "__main__":
27     print("Prefiero ser un módulo, pero puedo hacer algunas pruebas pa
28     my_list = [i+1 for i in range(5)]
29     print(suml(my_list) == 15)
30     print(prodl(my_list) == 120)
31
```

Algunos elementos necesitan explicación:

- La línea que comienza con `#!` tiene muchos nombres - puede ser llamada *shabang*, *shebang*, *hashbang*, *poundbang* o incluso *hashpling* (no nos preocupemos por qué). El nombre en sí no significa nada aquí, su papel es más importante. Desde el punto de vista de Python, es solo un **comentario** debido a que comienza con `#`. Para sistemas operativos Unix y similares a Unix (incluido MacOS), dicha línea **indica al sistema operativo como ejecutar el contenido del archivo** (en otras palabras, que programa debe ejecutarse para interpretar el texto). En algunos entornos (especialmente aquellos conectados con servidores web) la ausencia de esa línea causará problemas.
- Una cadena (quizás una multilínea) colocada antes de las instrucciones de cualquier módulo (incluidas las importaciones) se denomina **doc-string**, y debe explicar brevemente el propósito y el contenido del módulo.
- Las funciones definidas dentro del módulo (`sum1()` y `prod1()`) están disponibles para ser importadas.
- Se ha utilizado la variable `__name__` para detectar cuando se ejecuta el archivo de forma independiente, y se aprovechó esta oportunidad para realizar algunas pruebas sencillas.

Paso 10

Ahora es posible usar el nuevo módulo, esta es una forma de hacerlo:

```

1  | from module import sum1, prod1
2
3  | zeroes = [0 for i in range(5)]
4  | ones = [1 for i in range(5)]
5  | print(sum1(zeroes))
6  | print(prod1(ones))
7

```

main.py

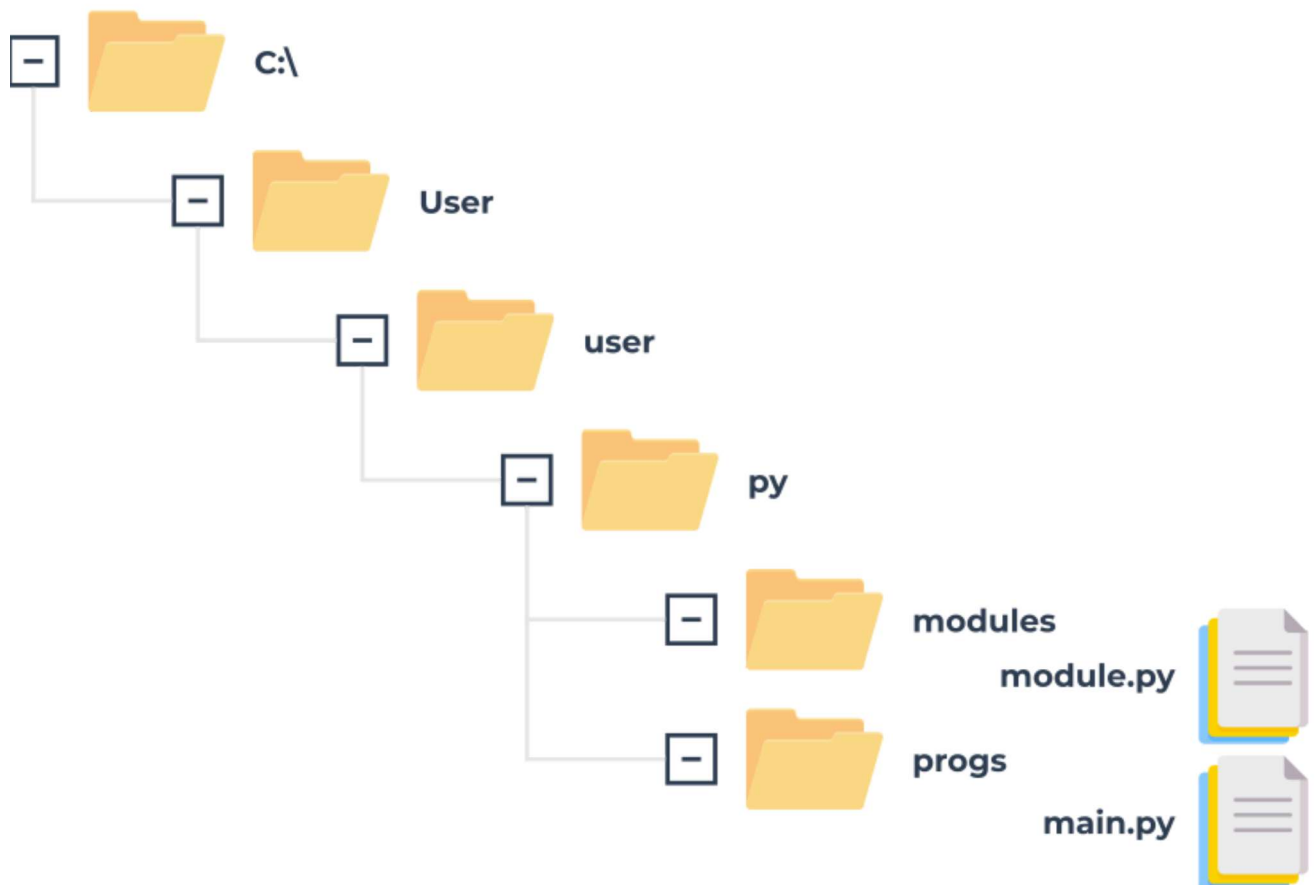
Paso 11

Es hora de hacer este ejemplo más complejo: hemos asumido aquí que el archivo Python principal se encuentra en la misma carpeta o directorio que el módulo que se va

a importar.

Renunciemos a esta suposición y realicemos el siguiente experimento mental:

- Estamos utilizando el sistema operativo Windows® (esta suposición es importante ya que la forma del nombre del archivo depende de ello).
- El script principal de Python se encuentra en `C:\Users\user\py\progs` y llama `main.py`.
- El módulo a importar se encuentra en `C:\Users\user\py\modules`



¿Cómo lidiar con ello?

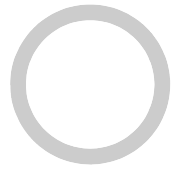
Para responder a esta pregunta, tenemos que hablar sobre **como Python busca los módulos**. Hay una variable especial (en realidad una lista) que almacena todas las ubicaciones (carpetas o directorios) que se buscan para encontrar un módulo que ha sido solicitado por la instrucción `import`.

Python examina estas carpetas en el orden en que aparecen en la lista: si el módulo no se puede encontrar en ninguno de estos directorios, la importación falla.

De lo contrario, se tomará en cuenta la primera carpeta que contenga un módulo con el nombre deseado (si alguna de las carpetas restantes contiene un módulo con ese nombre, se ignorará).

La variable se llama `path` (ruta), y es accesible a través del módulo llamado `sys`. Así es como puedes verificar su valor:

```
1 | import sys
2 |
3 | for p in sys.path:
4 |     print(p)
5 |
```



Hemos ejecutado el código dentro del directorio `C:\User\user` y esto es lo que obtenemos:

Output

```
C:\Users\user
C:\Users\user\AppData\Local\Programs\Python\Python36-32\python36.zip
C:\Users\user\AppData\Local\Programs\Python\Python36-32\lib
C:\Users\user\AppData\Local\Programs\Python\Python36-32
C:\Users\user\AppData\Local\Programs\Python\Python36-32\lib\site-packages
```

Nota: la carpeta en la que comienza la ejecución aparece en el **primer elemento de la ruta**.

Ten en cuenta también que: hay un archivo zip listado como uno de los elementos de la ruta, esto no es un error. Python puede tratar los archivos zip como carpetas ordinarias, esto puede ahorrar mucho almacenamiento.

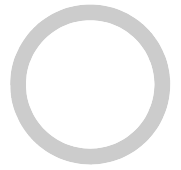
¿Puedes predecir cómo resolver este problema? Puedes resolverlo agregando una carpeta que contenga el módulo a la variable de ruta (la variable `path`), es completamente modificable.

Paso 12

Una de las varias soluciones posibles se ve así:

```
1 | from sys import path
2 |
3 | path.append('..\modules')
4 |
5 | import module
6 |
```

```
7 | zeroes = [0 for i in range(5)]
8 | ones = [1 for i in range(5)]
9 | print(module.suml(zeroes))
10 | print(module.prod1(ones))
11 |
```



main.py

Nota

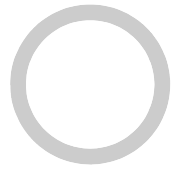
- Se ha duplicado la \ dentro del nombre de la carpeta, ¿sabes por qué?

Revisar

- Hemos utilizado el nombre relativo de la carpeta: esto funcionará si inicia el archivo `main.py` directamente desde la carpeta de inicio, y no funcionará si el directorio actual no se ajusta a la ruta relativa; siempre puedes usar una ruta absoluta, como esta:

```
path.append('C:\\Users\\user\\py\\modules')
```

- Hemos usado el método `append()`, la nueva ruta ocupará el último elemento en la lista de rutas; si no te gusta la idea, puedes usar en lugar de ello el método `insert()`.



1.3.3 Tu primer paquete

Paso 1

Imagina que en un futuro no muy lejano, tu y tus socios escriben una gran cantidad de funciones en Python.

Tu equipo decide agrupar las funciones en módulos separados, y este es el resultado final:

```
1  |  #! /usr/bin/env python3
2  |
3  |  """ module: alpha """
4  |
5  |  def funA():
6  |      return "Alpha"
7  |
8  |  if __name__ == "__main__":
9  |      print("Prefiero ser un módulo.")
10 |
```

alpha.py

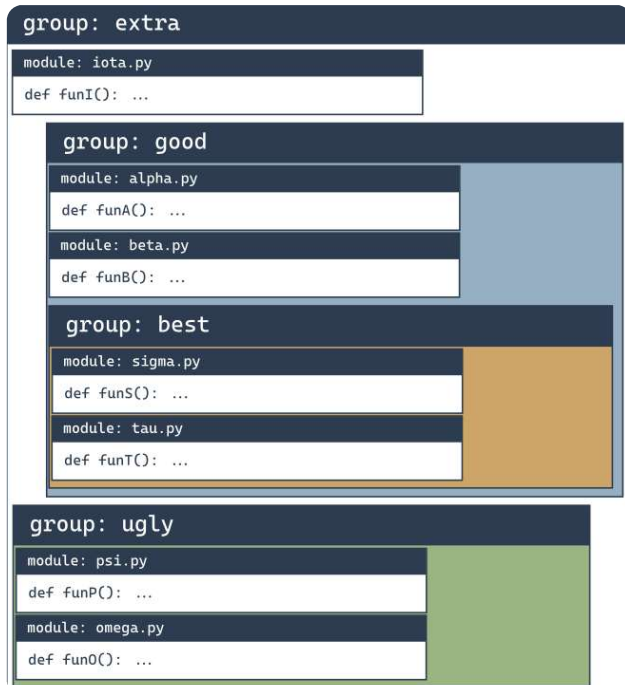
Nota: hemos presentado todo el contenido solo para el módulo `alpha.py`, supongamos que todos los módulos tienen un aspecto similar (contienen una función denominada `funX`, donde `X` es la primera letra del nombre del módulo).

alpha.py <pre>#!/usr/bin/env python3 """ module: alpha """ def funA(): return "Alpha" if __name__ == "__main__": print("I prefer to be a module.")</pre>	beta.py <pre>def funB(): ...</pre>
	iota.py <pre>def funI(): ...</pre>
	sigma.py <pre>def funS(): ...</pre>
tau.py <pre>def funT(): ...</pre>	psi.py <pre>def funP(): ...</pre>
omega.py <pre>def funO(): ...</pre>	

Paso 2

De repente, alguien se da cuenta de que estos módulos forman su propia jerarquía, por lo que colocarlos a todos en una estructura plana no será una buena idea.

Después de algo de discusión, el equipo llega a la conclusión de que los módulos deben agruparse. Todos los participantes están de acuerdo en que la siguiente estructura de árbol refleja perfectamente las relaciones mutuas entre los módulos:



Repasemos esto de abajo hacia arriba:

- El grupo **ugly** contiene dos módulos: **psi** y **omega**.
- El grupo **best** contiene dos módulos: **sigma** y **tau**.
- El grupo **good** contiene dos módulos: (**alpha** y **beta**) y un subgrupo (**best**).
- El grupo **extra** contiene dos subgrupos: (**good** y **bad**) y un módulo (**iota**).

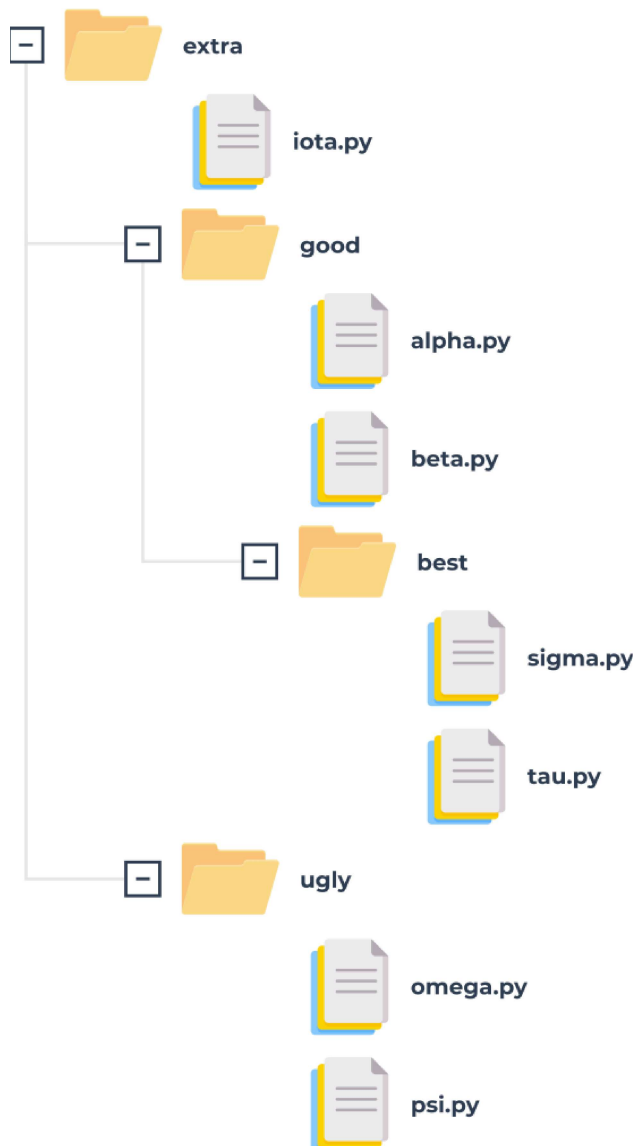
¿Se ve mal? De ninguna manera: analiza la estructura cuidadosamente. Se parece a algo, ¿no?

Parece la **estructura de un directorio**.

Construyamos un árbol que refleje las dependencias proyectadas entre los módulos.

Paso 3

Así es como se ve el árbol actualmente:



Tal estructura es casi un paquete (en el sentido de Python). Carece del detalle fino para ser funcional y operativo. Lo completaremos en un momento.

Si asumes que `extra` es el nombre de un **paquete recientemente creado** (piensa en él como la **raíz del paquete**), impondrá una regla de nomenclatura que te permitirá nombrar claramente cada entidad del árbol.

Por ejemplo:

- La ubicación de una función llamada `funT()` del paquete `tau` puede describirse como:

```
extra.good.best.tau.funT()
```

- Una función marcada como:

```
extra.ugly.psi.funP()
```

Proviene del módulo `psi` el cual está almacenado en el subpaquete `ugly` del paquete `extra`.

Paso 4

Ahora se deben responder dos preguntas:

- ¿**Cómo** se transforma este árbol (en realidad, un subárbol) en un **paquete** real de Python (en otras palabras, ¿cómo convence a Python de que dicho árbol no es solo un montón de archivos basura, sino un conjunto de módulos)?
- ¿**Dónde** se coloca el subárbol para que Python pueda acceder a él?

La primera pregunta tiene una respuesta sorprendente: **los paquetes, como los módulos, pueden requerir inicialización.**

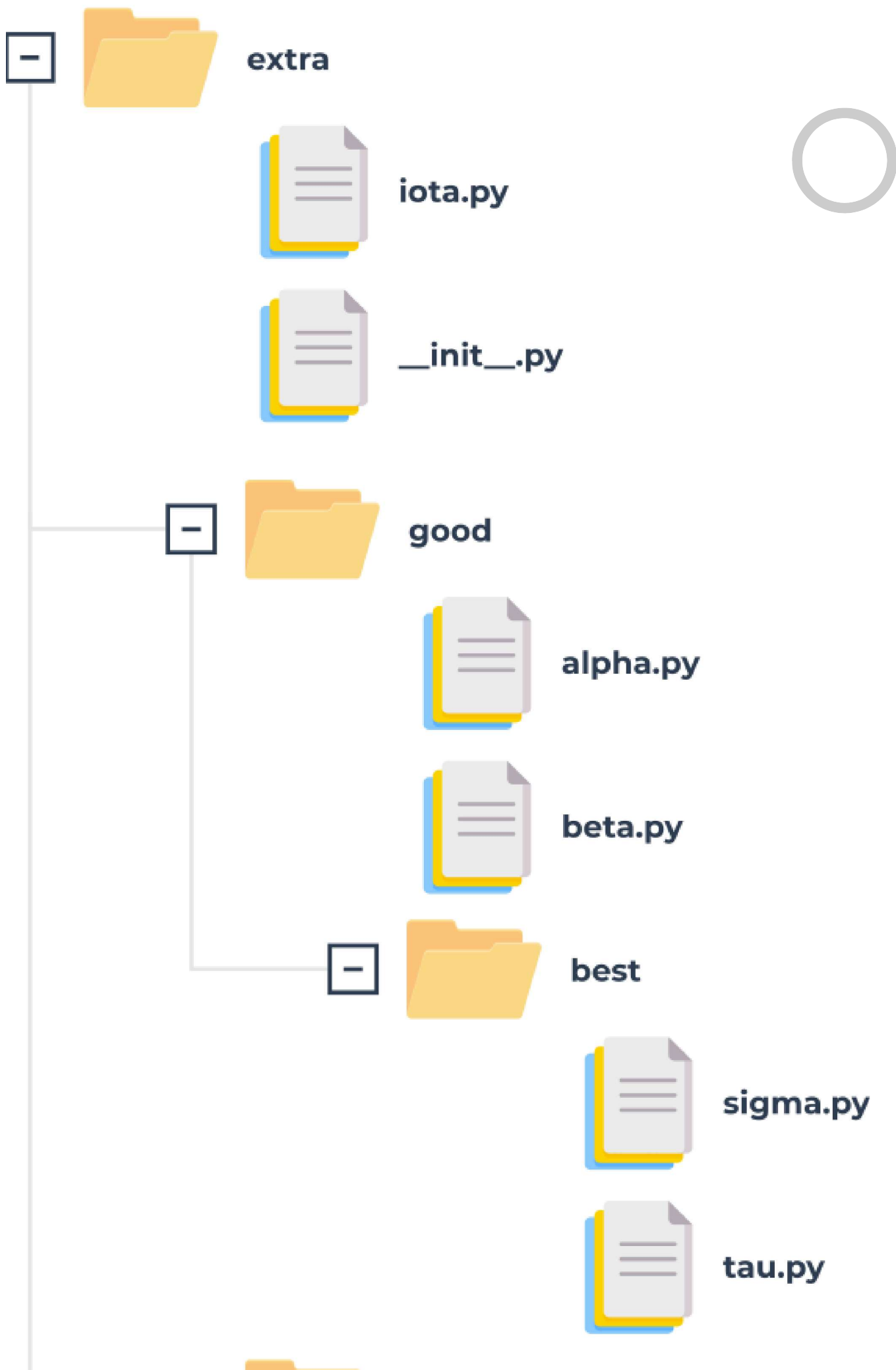
La inicialización de un módulo se realiza mediante un código *independiente* (que no forma parte de ninguna función) ubicado dentro del archivo del módulo. Como un paquete no es un archivo, esta técnica es inútil para inicializar paquetes.

En su lugar, debes usar un truco diferente, Python espera que haya un archivo con nombre muy exclusivo dentro de la carpeta del paquete: `__init__.py`.

El contenido del archivo se ejecuta cuando se **importa** cualquiera de los módulos del paquete. Si no deseas ninguna inicialización especial, puedes dejar el archivo vacío, pero no debes omitirlo.

Paso 5

Recuerda: **la presencia del archivo `__init__.py` finalmente compone el paquete:**





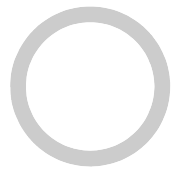
Nota: no solo la carpeta *raíz* puede contener el archivo `__init__.py`, también puedes ponerlo dentro de cualquiera de sus subcarpetas (subpaquetes). Puede ser útil si algunos de los subpaquetes requieren tratamiento individual o un tipo especial de inicialización.

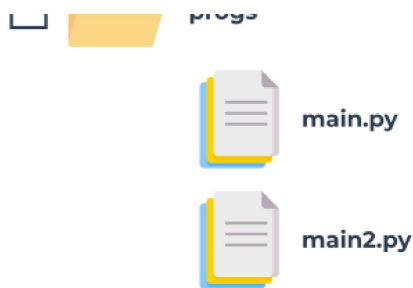
Ahora es el momento de responder la segunda pregunta, ¿Dónde se coloca el subárbol para que sea accesible para Python? La respuesta es simple: **donde quiera**. Solo tienes que asegurarte de que Python conozca la ubicación del paquete. Ya sabes como hacer eso.

Estás listo para usar tu primer paquete.

Paso 6

Supongamos que el entorno de trabajo se ve de la siguiente manera:





Hemos preparado un archivo zip que contiene todos los archivos de la rama de paquetes. Puedes descargarlo y usarlo para tus propios experimentos, pero **recuerda desempaquetarlo en la carpeta presentada en el esquema**, de lo contrario, no será accesible para el código.

[DESCARGAR](#) [Archivo ZIP Módulos y Paquetes](#)

Continuarás tus experimentos empleado el archivo `main2.py`.

Paso 7

Vamos a acceder a la función `funI()` del módulo `iota` del paquete `extra`. Nos obliga a usar nombres de paquetes calificados (asocia esto al nombramiento de carpetas y subcarpetas).

Así es como se hace:

```
1 | from sys import path
2 | path.append('../\packages')
3 |
4 | import extra.iota
5 | print(extra.iota.funI())
6 |
```

`main2.py`

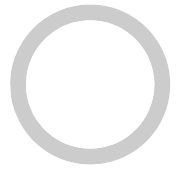
Nota:

- Hemos modificado la variable `path` para que sea accesible a Python.
- El `import` no apunta directamente al módulo, pero especifica la ruta completa desde la parte superior del paquete.

El reemplazar `import extra.iota` CON `import iota` causará un error.

La siguiente variante también es válida:

```
1 | from sys import path
2 | path.append('../\\packages')
3 |
4 | from extra.iota import funI
5 | print(funI())
6 |
```



main.2.py

Nota el nombre calificado del módulo `iota`.

Paso 8

Ahora vamos hasta el final del árbol: así es como se obtiene acceso a los módulos `sigma` y `tau`:

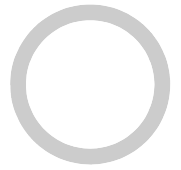
```
1 | from sys import path
2 |
3 | path.append('../\\packages')
4 |
5 | import extra.good.best.sigma
6 | from extra.good.best.tau import funT
7 |
8 | print(extra.good.best.sigma.funS())
9 | print(funT())
10 |
```

main2.py

Puedes hacer tu vida más fácil usando un alias:

```
1 | from sys import path
2 |
3 | path.append('../\\packages')
4 |
```

```
5
6 import extra.good.best.sigma as sig
7 import extra.good.alpha as alp
8
9 print(sig.funS())
10 print(alp.funA())
11
```



main2.py

Paso 9

Supongamos que hemos comprimido todo el subdirectorio, comenzando desde la carpeta `extra` (incluyéndola), y se obtuvo un archivo llamado `extrapack.zip`. Después, colocamos el archivo dentro de la carpeta `packages`.

Ahora podemos usar el archivo zip en un rol de paquetes:

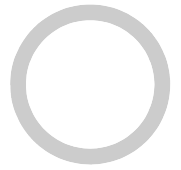
```
1 from sys import path
2
3 path.append('../\\packages\\extrapack.zip')
4
5 import extra.good.best.sigma as sig
6 import extra.good.alpha as alp
7 from extra.iota import funI
8 from extra.good.beta import funB
9
10 print(sig.funS())
11 print(alp.funA())
12 print(funI())
13 print(funB())
14
```

main2.py

Si deseas realizar tus propios experimentos con el paquete que hemos creado, puedes descargarlo a continuación. Te alentamos a que lo hagas.

DESCARGAR [Archivo ZIP Extrapack](#)

Ahora puedes crear módulos y combinarlos en paquetes. Es hora de comenzar una discusión completamente diferente: sobre errores y fallas.



1.3.4 RESUMEN DE SECCIÓN

1. Mientras que un **módulo** está diseñado para acoplar algunas entidades relacionadas como funciones, variables o constantes, un **paquete** es un contenedor que permite el acoplamiento de varios módulos relacionados bajo un mismo nombre. Dicho contenedor se puede distribuir tal cual (como un lote de archivos implementados en un subárbol de directorio) o se puede empaquetar dentro de un archivo zip.
2. Durante la primera importación del módulo, Python traduce su código fuente a un formato **semi-compilado** almacenado dentro de los archivos **pyc** y los implementa en el directorio `__pycache__` ubicado en el directorio de inicio del módulo.
3. Si deseas decirle al usuario del módulo que una entidad en particular debe tratarse como **privada** (es decir, no debe usarse explícitamente fuera del módulo), puedes marcar su nombre con el prefijo `_` o `__`. No olvides que esto es solo una recomendación, no una orden.

4. Los nombres *shabang*, *shebang*, *hasbang*, *poundbang* y *hashpling* describen el dígrafo escrito como `#!`, se utiliza para instruir a los sistemas operativos similares a Unix sobre cómo se debe iniciar el archivo fuente de Python. Esta convención no tiene efecto en MS Windows.

5. Si deseas convencer a Python de que debe tomar en cuenta el directorio de un paquete no estándar, su nombre debe insertarse/agregarse en/a la lista de directorios de importación almacenada en la variable `path` contenida en el módulo `sys`.

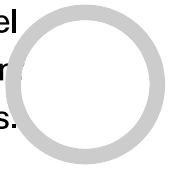
6. Un archivo de Python llamado `__init__.py` se ejecuta implícitamente cuando un paquete que lo contiene está sujeto a importación y se utiliza para inicializar un paquete y/o sus subpaquetes (si los hay). El archivo puede estar vacío, pero no debe faltar.

1.3.5 CUESTIONARIO DE SECCIÓN

Pregunta 1: Deseas evitar que el usuario de tu módulo ejecute tu código como un script ordinario. ¿Cómo lograrías tal efecto?

Revisar

Pregunta 2: Algunos paquetes adicionales y necesarios se almacenan dentro del directorio `D:\Python\Project\Modules` directory. Escribe un código asegurándote de que Python recorra el directorio para encontrar todos los módulos solicitados.



Revisar

Pregunta 3: El directorio mencionado en el Pregunta anterior contiene un subárbol con la siguiente estructura:

```
abc
|__ def
    |__ mymodule.py
```

Asumiendo que `D:\Python\Project\Modules` se ha adjuntado con éxito a la lista `sys.path` , escribe una directiva de importación que te permita usar todas las entidades de `mymodule` .

Revisar